

- Dejo el [repo](#) del equipo con todo el código que hemos trabajado durante el semestre
 - principalmente usamos un estructura modelo-control-vista (MVC), por lo que nos permitió tener una organización de flujos tanto backend como frontend, y su conexión entre ambos.
 - La modularización de nuestras rutas era principalmente para evitar el famoso spaghetti code, igual para evitar archivos tan extensos donde se pueden perder implementaciones de diferentes funciones. Esto también nos ayuda separar rutas por rol, ya que por ejemplo oficial tenía '/oficial' en su ruta con ciertas funcionalidades y tenía varias páginas mientras que el operador tenía otras.

El pair-programming es más difícil porque no hay pruebas. Como equipo tratamos de trabajar en el mismo lugar para poder preguntar cualquier duda. Definitivamente tuve más oportunidades de hacer esto con mi compañero German, que por ratos el veía como implementar ciertas cosas (como las tablas que la mayoría las hice yo) para que el revisara los errores pero también para entender cómo puede afectar él estas funciones al momento de implementar otras funciones. Igual yo en momentos veía como construye la bitácora ya que yo también iba a implementarla. Con mi compañero Oscar hubo momentos en donde nos sentamos a ver el código de uno o el otro para validar flujos o errores para seguir la consistencia del proyecto y evitar errores.

```

router.get('/contratos', contratos.index);
router.get('/operaciones', operaciones.index);
router.get('/listas', listas.index);
router.get('/reportes', reportes.index);

router.get('/kyc', (req, res) =>
  res.render('oficial/forms/kyc-form', {
    pageTitle: 'Nuevo cliente' }));
router.post('/new-client', (req, res) =>
  res.redirect('/oficial/dashboard'));
router.get('/registrar-operacion', operaciones.getRegistrarOperacion);
router.post('/new-operacion', operaciones.postRegistrarOperacion);
router.get('/registrar-contrato', contratos.getRegistrarContrato);
router.post('/new-contrato', contratos.postRegistrarContrato);
router.get('/evaluar-reporte', (req, res) =>
  res.render('oficial/forms/evaluar-caso-form', {
    pageTitle: 'Forma de evaluación' }));
router.post('/evaluar-caso', canalInterno.evaluation);
router.get('/reportes/generar', reportes.dailyReport);

//return the data
router.get('/operaciones/operacionesCount', operaciones.count);
router.get('/operaciones/operacionesData', operaciones.operaciones);
router.get('/dashboard/api/alertasDataDashboard', dashboard.alertas);
router.get('/dashboard/api/clientesDataDashboard', dashboard.clientes);
router.get('/dashboard/api/operacionesDataDashboard', dashboard.operaciones);
router.get('/alertas/api/alertasData', alertas.getAlertasData);
router.get('/contratos/contratosCount', contratos.count);
router.get('/contratos/contratosData', contratos.contratos);
router.get('/canalInterno/canalInternoData', canalInterno.getCanalInternoData);

// Page routes - protected by session login + role check
router.use('/oficial', requireLogin, requireRol(['oficial']), require('./oficial'));
router.use('/admin', requireLogin, requireRol(['admin']), require('./admin'));
router.use('/operador', requireLogin, requireRol(['operador']), require('./operador'));
router.use('/cliente', requireLogin, requireRol(['cliente']), require('./cliente'));

// Auth routes - no protection (login page must always be reachable)
router.use('/usuarios', require('./usuarios'));

// API routes - protected per-route via header-based auth (middleware/auth.js)
router.use('/api/clientes', require('./api/clientes'));
router.use('/api/expediente', require('./api/expedientes'));

//end session
router.use('/logout', usuarios.logout);

module.exports = router;

```

Por ejemplo, aquí se ve la diferencia entre nuestra organización de rutas. Se puede decir que es tipo jerárquico, donde lo de oficial (primera foto) pasa por las rutas en general (segunda foto). Esto evita estar escribiendo /oficial en cada ruta en un index.

```

const testTable = document.getElementById("test-table");

window.addEventListener("load", () => {
  gridTable = new gridjs.Grid({
    columns: ["Folio",
      "Descripción",
      "Fecha",
      "Estatus",
      { name: "Evidencia",
        formatter: (_, row) => {
          return gridjs.html(`<a href="/oficial/evidencia/${row.cells[0].data}">Ver evidencia</a>`)}
      }
    ],
    sort: true,
    pagination: {
      limit: 5,
      summary: false
    },
    server: {
      url: '/oficial/canalInterno/canalInternoData',
      then: data => data.map(ri => [
        ri.idreporteint,
        ri.descripcion,
        new Date(ri.fecha).toLocaleDateString('es-MX', {
          day: '2-digit',
          month: '2-digit',
          year: 'numeric'
        })
      ]),
      className: {

```

Igual una vista rápida al código de una tabla dentro de la página canal interno. Esto es un script aparte que afecta el front-end (vista), pero tiene varios factores que lo conectan con el back end para poder desplegar información en tiempo real. Los datos vienen del controlador que agarra queries del modelo.

- al momento de trabajar en conjunto como equipo, al inicio era difícil entender como trabajar sin que afectamos el trabajo de alguien más o que hagamos algo que otro compañero ya estaba trabajando. Para esto tuvimos que crear mucha estructura en nuestra forma de trabajar y realmente entender como usar github en todo su alcance (no era de solo crear un repositorio y subir archivos directo).

footer and dashboard rework #10

 Merged [greziatrujillo](#) merged 4 commits into [development](#) from [test-edits](#)  yesterday

 Conversation **1**  Commits **4**  Checks **1**  Files changed **21**



[greziatrujillo](#) commented [yesterday](#) Collaborator 

minor changes to dashboard page. Footer was moved to a different container so that it fits the page better and adjusts depending on the sidebar collapse.



 [greziatrujillo](#) added 4 commits [yesterday](#)

  [added badge colors on tables](#) 09a03a3

por ejemplo trabajamos mucho con ramas y pull requests. Aunque me hubiera gustado que los usáramos más y que nos diéramos el tiempo de comentar los códigos y tener reviews, pero vemos que eso no lo hicimos. Eso si es algo que nos sirve implementar en el futuro para que todos del equipo estén al tanto de los cambios, aunque solo por encima. Muchos merges después los hicimos localmente, por lo que dejamos de usar PR por un tiempo.